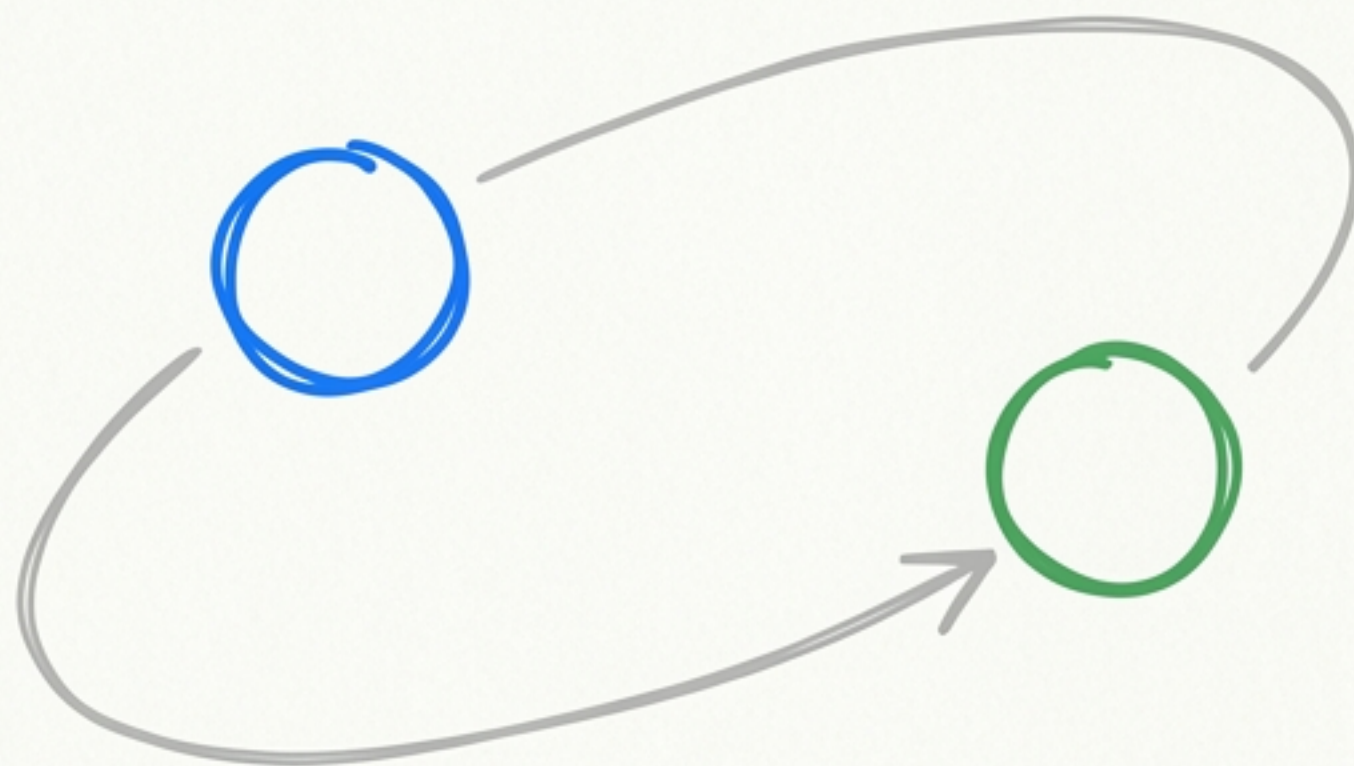


对比学习MOCO：无监督学习的动量之舞

Momentum Contrast for Unsupervised Visual Representation Learning



本次探索之旅的路线图



1. **基础篇**：什么是“对比”为核心的学习？



2. **挑战篇**：对比学习面临的核心困境



3. **破局篇**：MoCo如何巧妙破局？

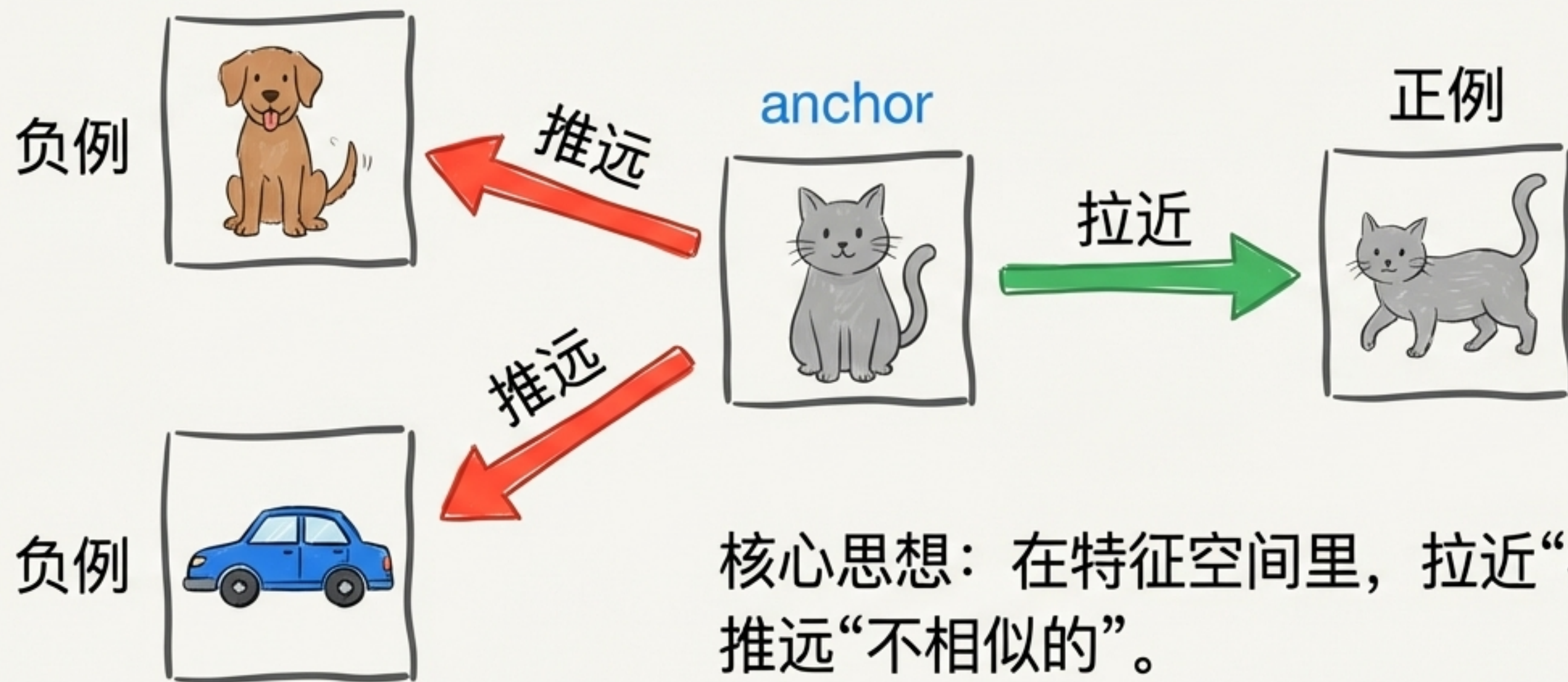


4. **深入篇**：深入MoCo架构与代码实现



5. **总结篇**：核心贡献与展望

什么是对比学习？目标不是分类，而是学习“表示”

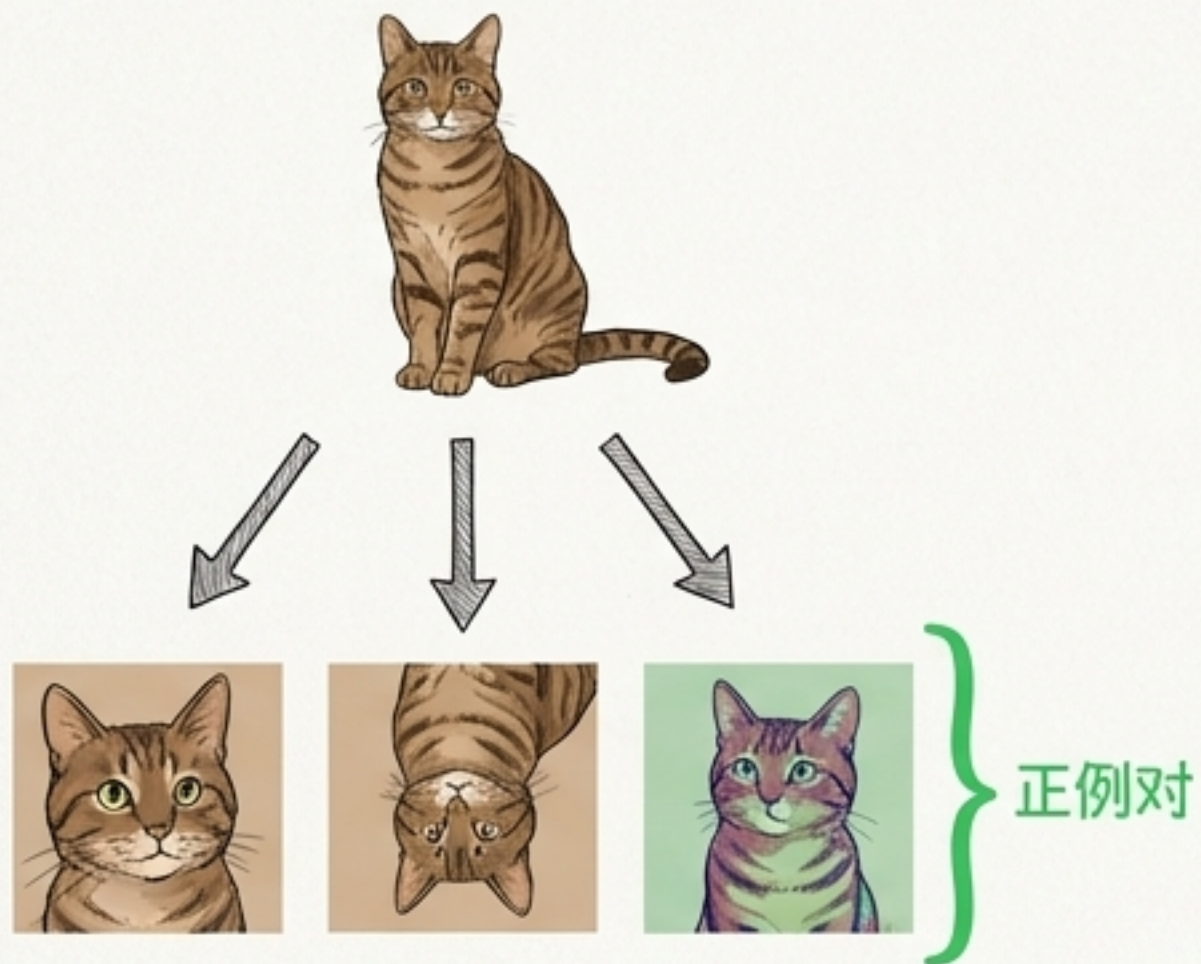


- 一种**无监督学习**，通过正例、负例，学习样本的优质特征表示。
- 学到的特征表示，在下游任务上简单微调就可以取得好结果。

“无中生有”：如何自动构建正负样本对？

我们自己就是“数据标注员”：通过数据增强，为一张图片创造出属于它自己的正样本。
任何其他的图片，都可视为负样本。

计算机视觉（实例判别）



自然语言处理（回译增强）

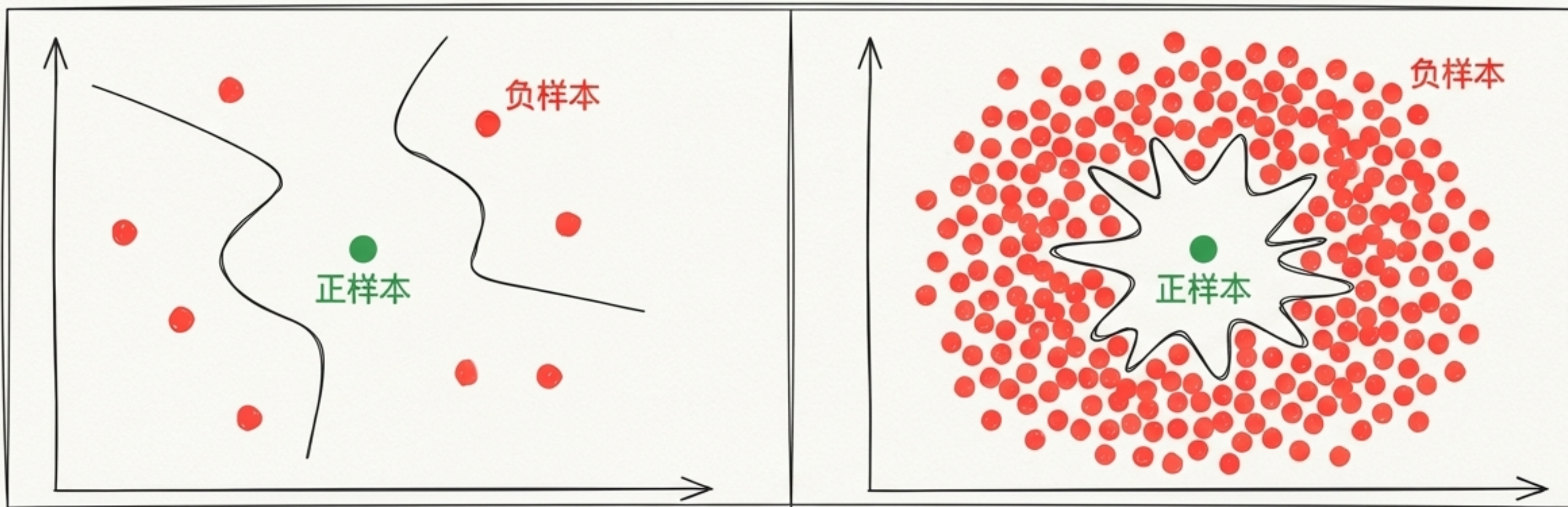


多模态（图文对）



对比学习的难点：为什么需要海量的负样本？

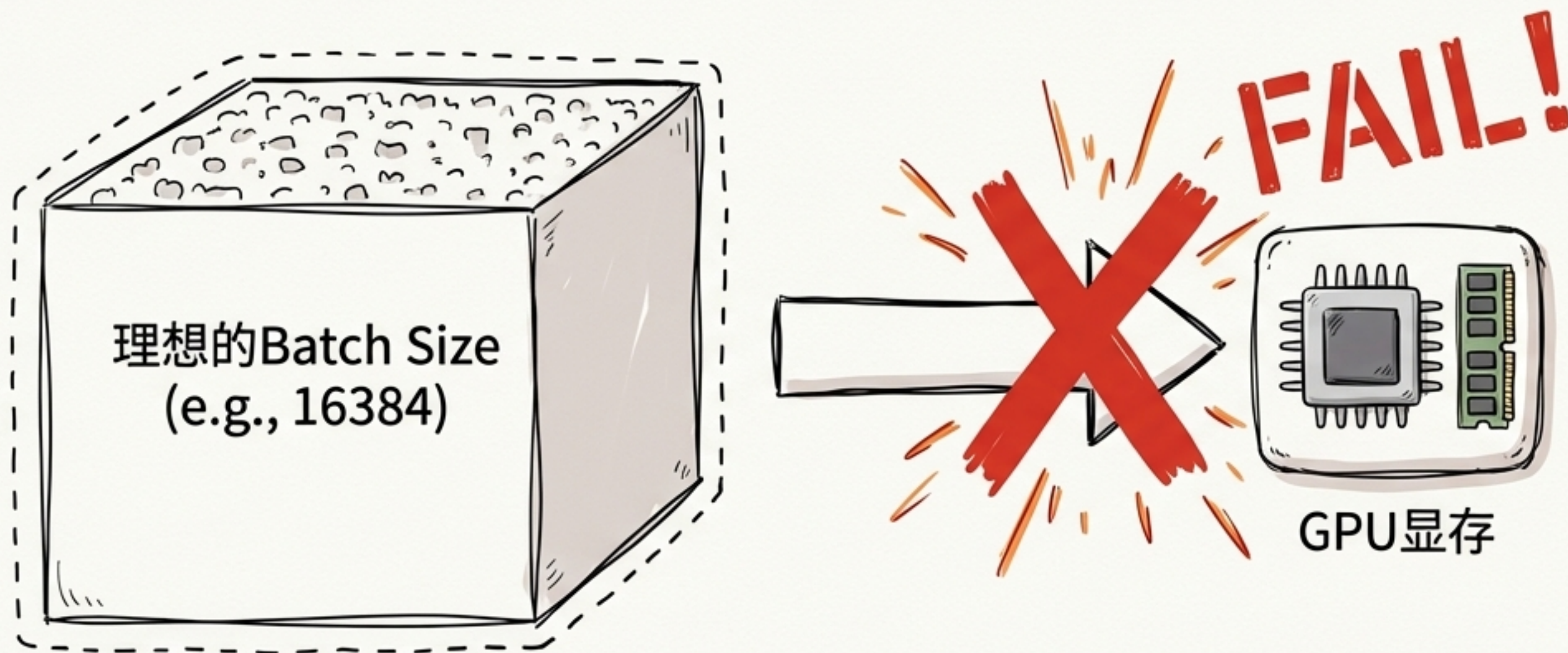
负样本的数量和多样性至关重要。如果负样本太少，模型会轻易地“记住”所有反例，而不是学习到真正具有泛化能力的特征。它需要在一个足够大的“人群”中辨认出自己的“朋友”。



负样本太少，学习任务过于简单

海量负样本，迫使模型学习更优的特征

核心矛盾：负样本数量 vs. GPU显存



在常规的端到端实现中，负样本全部来自同一个batch。为了获得海量负样本，我们就需要一个巨大的Batch Size。但这会直接超出单个GPU甚至多个GPU的显存上限。

一个好的对比学习框架，需要满足两大需求



“大”：负样本的“字典”要足够大。

这样才能让模型见多识广，学到泛化能力强的特征。



“一致”：“字典”中所有样本的特征编码要尽可能一致。

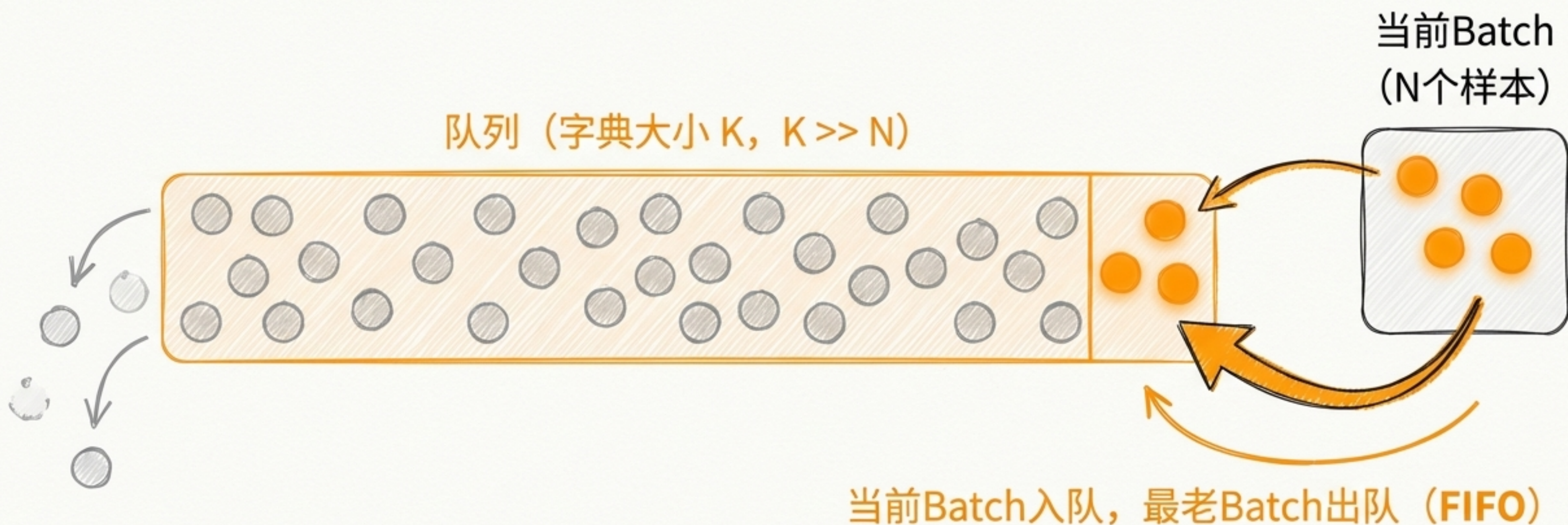
如果编码器在训练中变化太快，早先编码的负样本就失去了比较的意义，如同用一把变化的尺子测量物体。



尺子

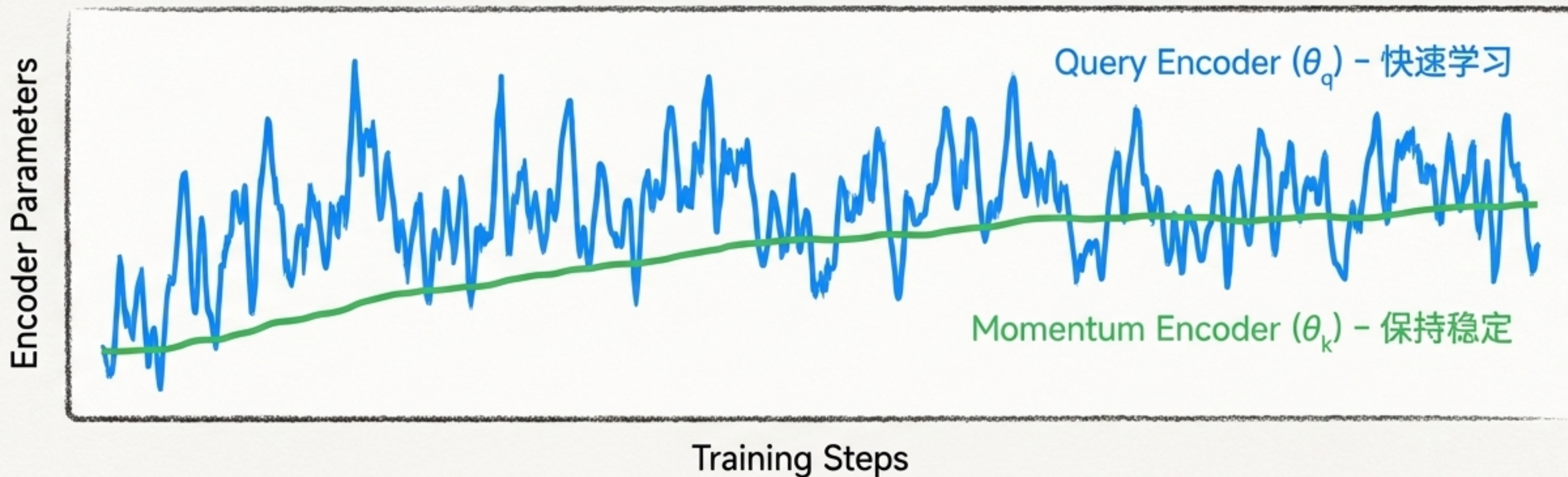
MoCo的巧思之一：用队列(Queue)解耦字典大小与Batch Size

MoCo不再让Batch Size决定负样本数量。它引入了一个独立的队列作为“动态字典”。这个队列可以存储远超Batch Size数量的负样本特征。



MoCo的巧思之二：用动量(Momentum)更新来保证“一致性”

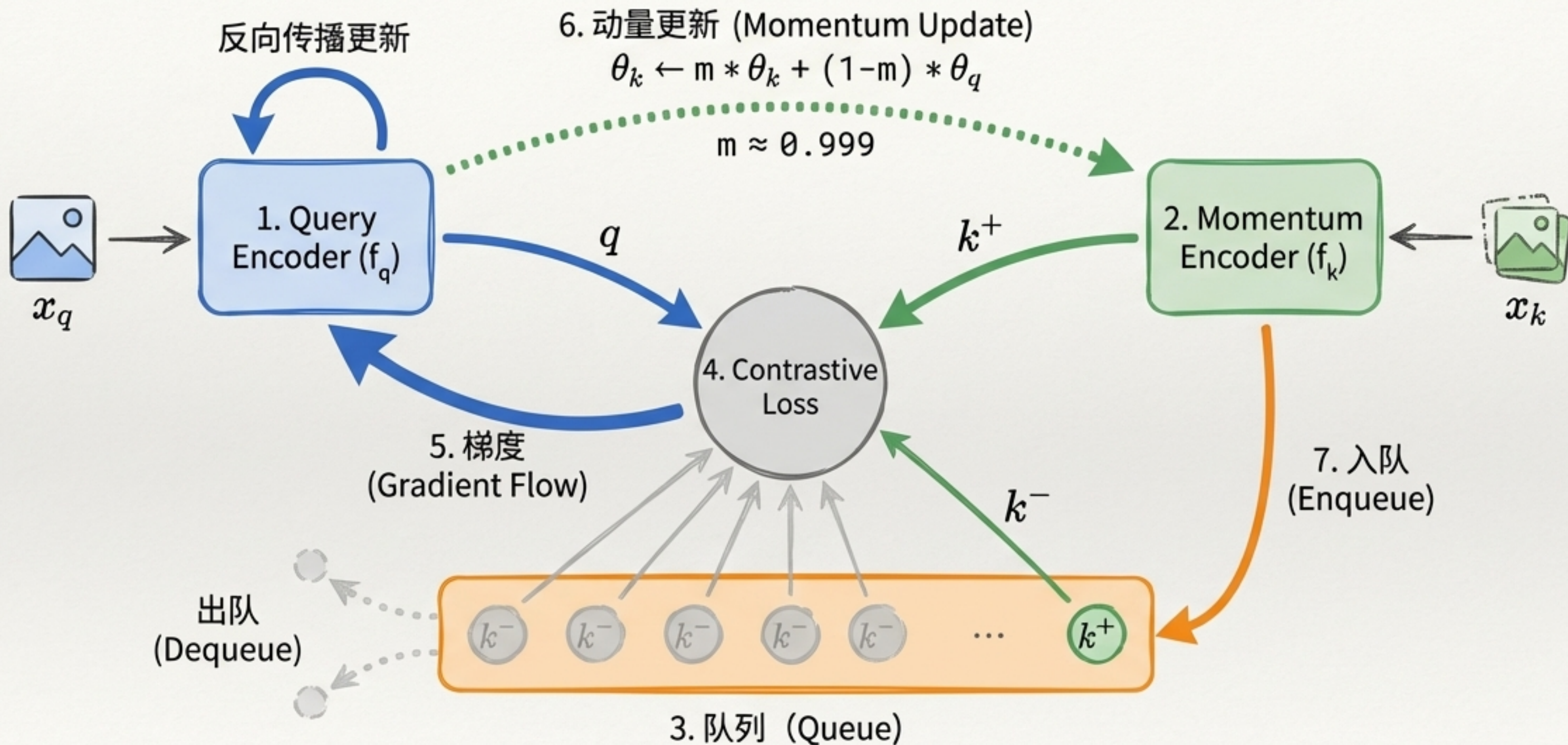
为了让队列中的“老”特征和“新”特征具有可比性，MoCo使用了两个编码器。查询编码器(Query Encoder)通过反向传播快速更新，而字典键编码器(Key Encoder)则采用动量方式进行极其缓慢、平滑的更新。



$$\theta_k \leftarrow m * \theta_k + (1-m) * \theta_q \longrightarrow$$

‘m’是一个很大的动量系数 (e.g., 0.999)。这意味着 θ_k 的更新非常缓慢，像一个平滑的移动平均，保证了字典特征的一致性。

MoCo完整工作流程



驱动引擎：InfoNCE损失函数

InfoNCE本质上是一个多分类交叉熵损失。它将任务看作是：给定一个查询 q ，在一个正样本 k^+ 和 K 个负样本 k^- 中，正确地将 q 分类到 k^+ 。

点积(Dot product)用于
衡量特征相似度。

温度系数 τ 用于调节
softmax的分布锐度。

$$\text{Loss} = -\log \left[\frac{\exp(\mathbf{q} \cdot \mathbf{k}_+ / \tau)}{\exp(\mathbf{q} \cdot \mathbf{k}_+ / \tau) + \sum \exp(\mathbf{q} \cdot \mathbf{k}_- / \tau)} \right]$$

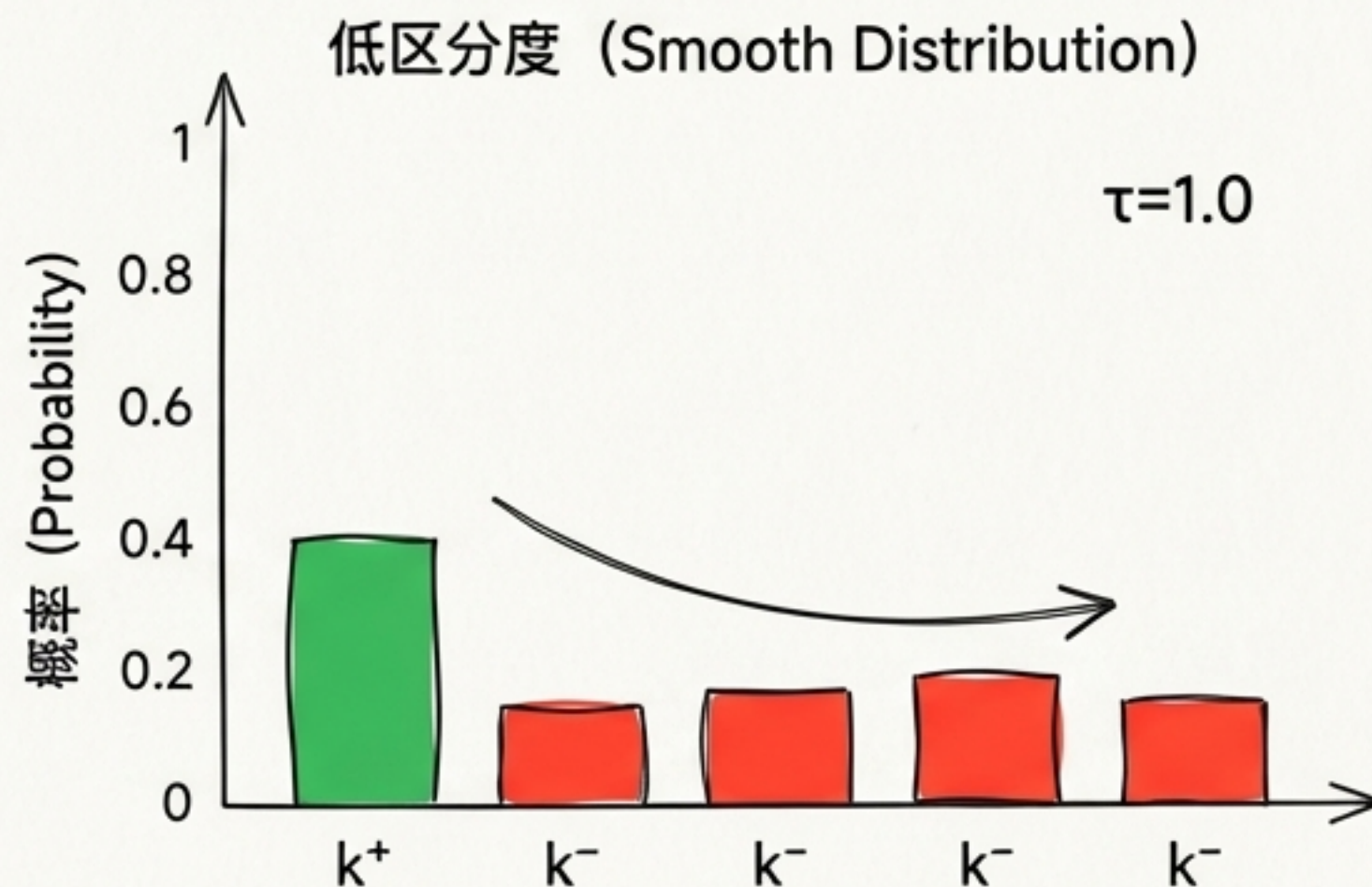
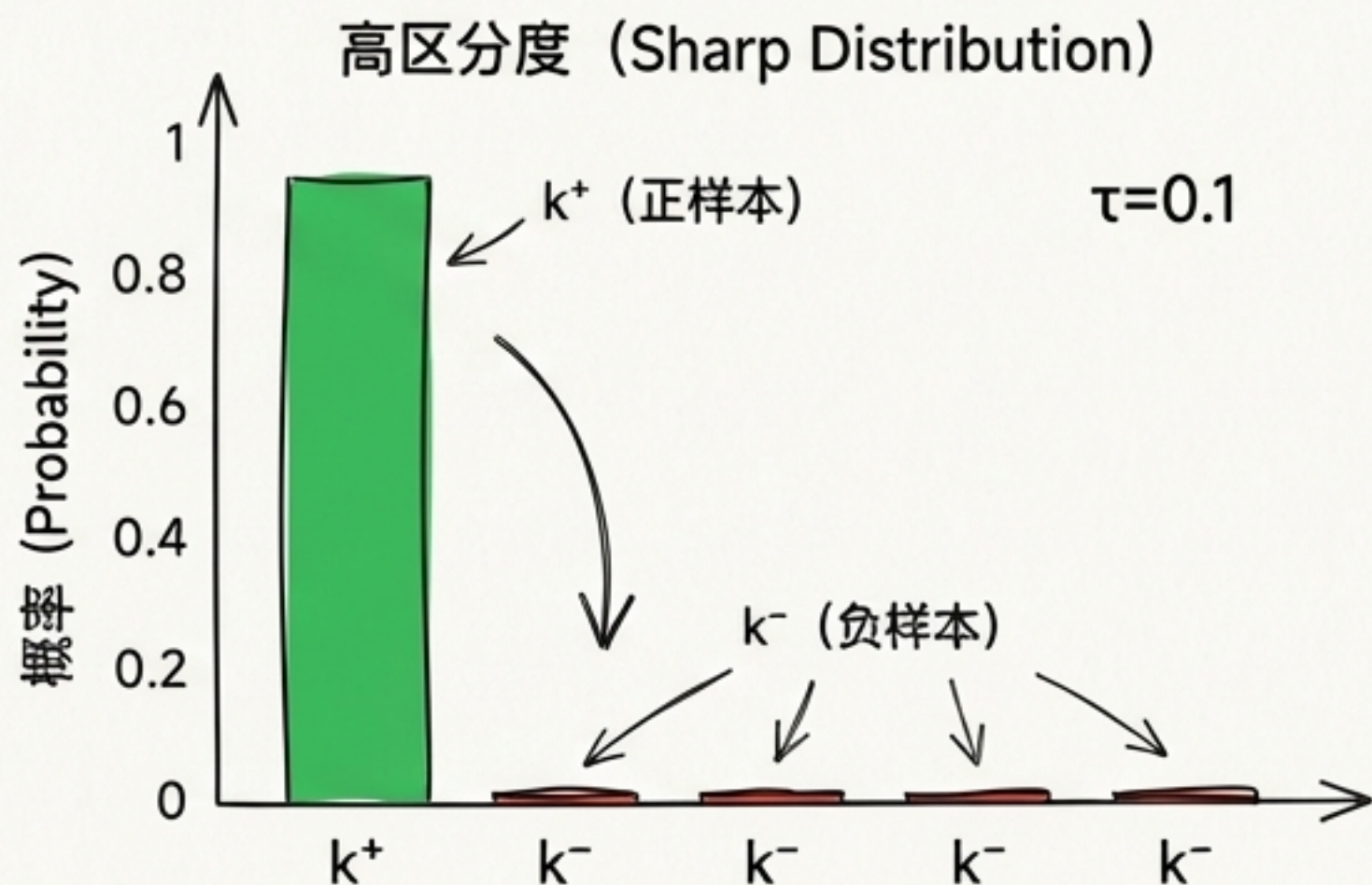
目标是最大化选中正
样本对的概率。

Softmax函数将相似度
分数转化为概率分布。

超参数调优：温度系数 τ 的作用

温度系数 τ 控制着模型对难分负样本的关注程度。

- τ 越小：Softmax分布越“尖锐”。模型必须将正样本的得分和所有负样本的得分都远远拉开，学习任务更难，但可能学得更好。
- τ 越大：Softmax分布越“平滑”。模型对得分差异不那么敏感，学习任务相对容易。



深入代码：MoCo的PyTorch风格伪代码实现

```
# f_q, f_k: encoder networks
# queue: dictionary as a queue
f_k.params = f_q.params # initialize

for x in loader: # load a minibatch
    x_q = aug(x) # a query view
    x_k = aug(x) # a key view

    # compute query features
    q = f_q(x_q) # queries: Nx C

    # compute key features
    with torch.no_grad(): # no gradient to keys
        k = f_k(x_k) # keys: Nx C

    # compute logits
    # positive logits: Nx1
    l_pos = bmm(q.view(N,1,C), k.view(N,C,1)) #
    # negative logits: NxK
    l_neg = mm(q.view(N,C), queue.view(C,K)) #

    # logits: Nx(1+K)
    logits = cat([l_pos, l_neg], dim=1)

    # contrastive loss
    loss = cross_entropy(logits, labels)

    # SGD update: query network
    loss.backward()
    update(f_q.params)

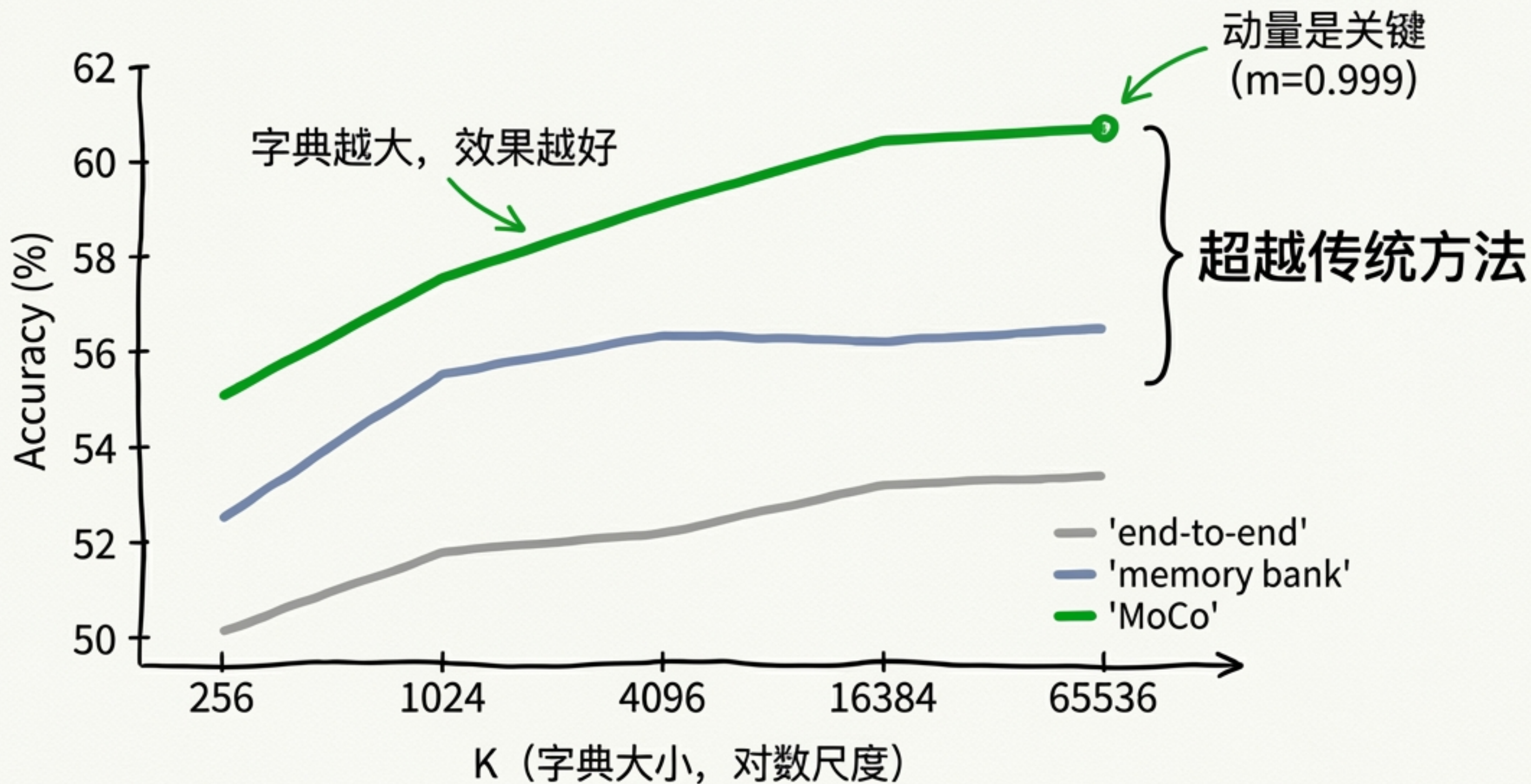
    # momentum update: key network
    f_k.params = m * f_k.params + (1-m) * f_q.params

    # update dictionary
    enqueue(queue, k)
```

← 关键一步！停止对key encoder的梯度回传。

← 动量更新逻辑在梯度计算之外执行。

实验为证：MoCo的效果



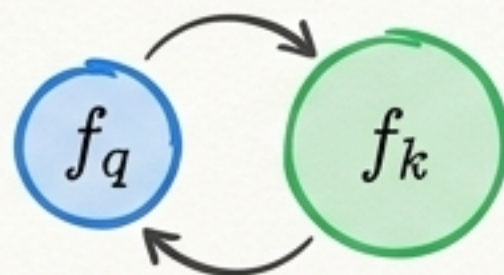
总结：MoCo的核心贡献与深远影响

队列 (Queue)



巧妙地解耦了字典大小和批次大小，实现了大批量负样本的对比学习，解决了“大”的问题。

动量编码器 (Momentum Encoder)



保证了字典中特征的一致性，解决了“一致”的问题。

范式创新



提出了一种高效且极为有效的无监督表示学习新范式，为后续的SimCLR, BYOL等工作奠定了基础。

深远影响

MoCo及其后续工作极大地推动了自监督学习的发展，使其在诸多下游任务（如目标检测、分割）上的表现，能够媲美甚至超越监督学习的预训练模型。

